EX 9 : TEXT GENERATION USING LSTM NETWORKS

AIM:

To Build a text generation model using Long Short-Term Memory (LSTM) networks. Train the model on a text corpus to generate coherent sequences of text and evaluate the output for fluency and coherence.

OBJECTIVES:

1. Understand sequential modeling for natural language generation.
2. Train a character-level LSTM model to learn language patterns.
3. Generate text using a seed prompt and evaluate the results.
4. Analyze the fluency and creativity of LSTM-generated outputs.

ALGORITHM:

1. Import the required libraries, including TensorFlow, Keras, and NumPy.

2. Download and load the Tiny Shakespeare text corpus and convert all characters to lowercase.

3. Identify all unique characters and create character-to-index and index-to-character mappings.

4. Create input sequences of 40 characters and use the next character as the target output.

5. Convert the input sequences and target characters into NumPy arrays.

6. Build an LSTM model using an embedding layer, an LSTM layer with 128 units, and a softmax output layer.

7. Compile the model using the Adam optimizer and sparse categorical cross-entropy loss.

8. Train the model on the text sequences for one epoch with a batch size of 128.

9. Define a text-generation function that accepts a seed prompt and the required output length.

10. Predict the next character using the trained model and randomly select a character based on the predicted probabilities.

11. Repeat the prediction process to generate the required number of characters.

12. Display the generated text and evaluate its fluency and coherence.

SOURCE CODE:

```
# Step 1: Load and Preprocess the Dataset
import tensorflow as tf
import numpy as np
text_path = tf.keras.utils.get_file(
    'shakespeare.txt',
```

```
    'https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt'
)

with open(text_path, 'r', encoding='utf-8') as file:
    text = file.read().lower()

chars = sorted(set(text))
c2i = {c: i for i, c in enumerate(chars)}
i2c = {i: c for i, c in enumerate(chars)}
print("Total characters:", len(text))
print("Unique characters:", len(chars))

# Step 2: Create Input and Output Sequences

seq_len = 40
X = []
y = []
for i in range(len(text) - seq_len):
    input_seq = text[i:i + seq_len]
    target_char = text[i + seq_len]
    X.append([c2i[c] for c in input_seq])
    y.append(c2i[target_char])
X = np.array(X, dtype=np.int32)
y = np.array(y, dtype=np.int32)
print("Input shape:", X.shape)
print("Output shape:", y.shape)

# Step 3: Build and Train the LSTM Model

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(seq_len,)),
    tf.keras.layers.Embedding(
        input_dim=len(chars),
        output_dim=64
    ),
    tf.keras.layers.LSTM(128),
    tf.keras.layers.Dense(
        len(chars),
        activation='softmax'
    )
])
```

```
model.compile(
    loss='sparse_categorical_crossentropy',
    optimizer='adam'
)
model.summary()
model.fit(
    X,
    y,
    batch_size=128,
    epochs=1
)

# Step 4: Define the Text Generation Function

def generate(seed, length=300):
    seed = seed.lower()
    # Handle seed characters not present in the dataset
    for c in seed:
        if c not in c2i:
            raise ValueError(
                f"Character {repr(c)} is not in the dataset vocabulary."
            )

    seq = [c2i[c] for c in seed]
    generated = []
    for _ in range(length):
        inp = np.array(
            seq[-seq_len:],
            dtype=np.int32
        ).reshape(1, -1)
        pred = model.predict(inp, verbose=0)[0]

        # Sample the next character
        next_idx = np.random.choice(
            len(pred),
            p=pred / pred.sum()
        )
        seq.append(next_idx)
        generated.append(i2c[next_idx])

    return seed + ''.join(generated)
```

```
# Step 5: Generate and Display Text

print("\nGenerated Text:\n")
print(
  generate(
    "shall i compare thee to a summer's day?\n",
    length=300
  )
)
```

OUTPUT:

```
Downloading data from https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
1115394/1115394 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Total characters: 1115394
Unique characters: 39
Input shape: (1115354, 40)
Output shape: (1115354,)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 40, 64)	2,496
lstm (LSTM)	(None, 128)	98,816
dense (Dense)	(None, 39)	5,031

```
Total params: 106,343 (415.40 KB)
Trainable params: 106,343 (415.40 KB)
Non-trainable params: 0 (0.00 B)
8714/8714 ━━━━━━━━━━━━━━━━━━━━ 1070s 122ms/step - loss: 1.8702

Generated Text:

shall i compare thee to a summer's day?

leoder:
if you becouch it, but mereep you was andry;
on in could nears,-- hadge! ever to which here i me
after.

lagius:
with your nithant best bake thou thinks;
these she couring of bokn him commive hasts;
apize; as i
pluatathin's come the tripst you corsunt.

colixio:
is heave ano have xeven grea
```

RESULT:

This experiment demonstrates how an LSTM model learns to predict and generate sequences of text based on character-level inputs. Despite being trained for a single epoch, the generated output showcases structural fluency and stylistic hints of Shakespearean English. Further training can improve coherence and creativity.